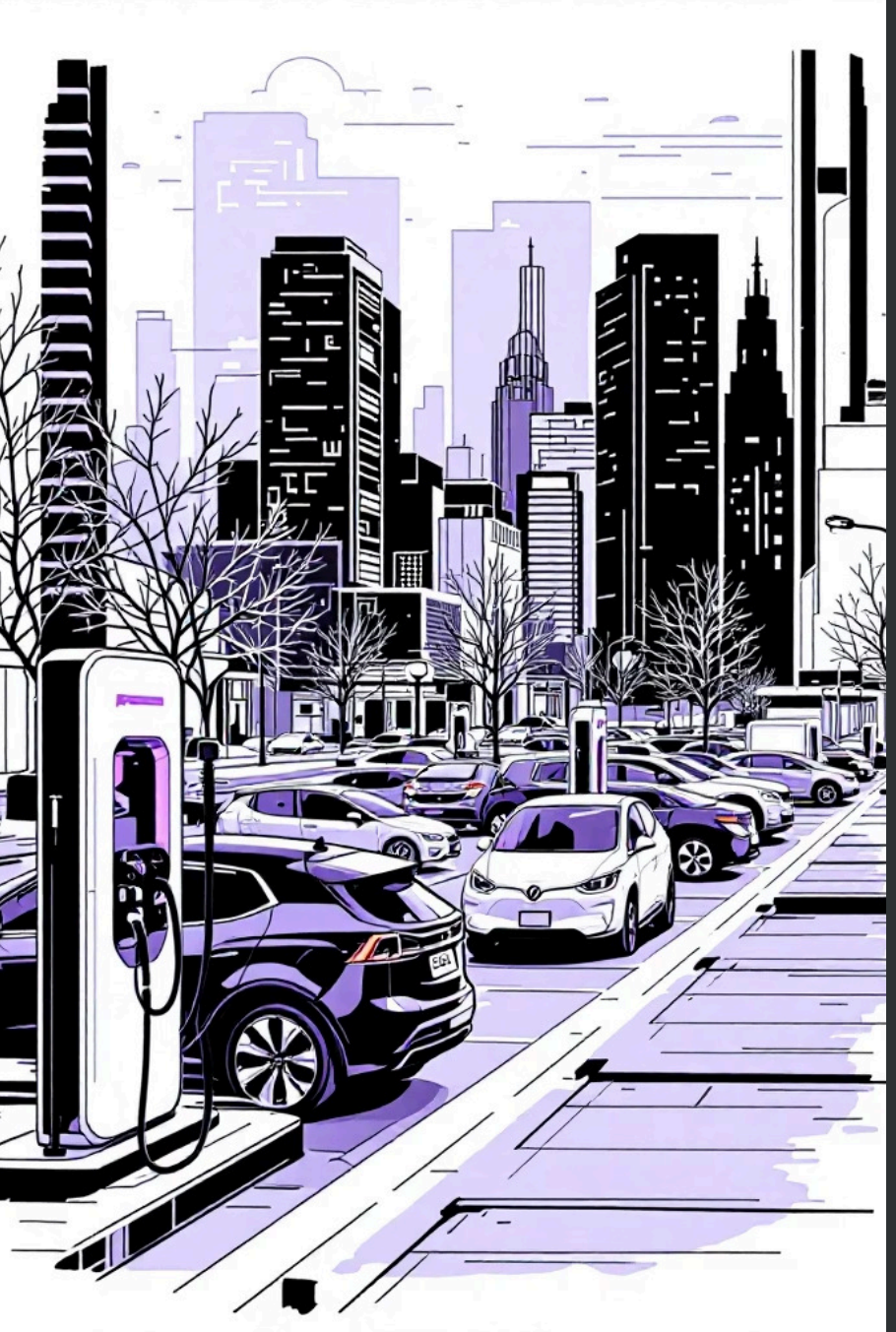
# Large Scale Data Project: EV Dataset

A complete hands-on project using **HDFS, MapReduce, Hive, Sqoop, Pig, Hue, HBase, PySpark, Kafka, and Cassandra** — processing real-world EV charging data end-to-end.

Made with GAMMA



# Dataset Overview

This dataset captures real-world EV charging sessions across a distributed network — recording energy consumption, session duration, location, and timestamps. It reflects the rapid growth of EV adoption and the operational complexity of managing charging infrastructure at scale.

### 🔋 Energy Tracking

kWh delivered per session, efficiency metrics, and cost analysis

### 📍 Location Intelligence

Geographic distribution of stations, urban vs rural usage patterns

### ⏱️ Usage Behavior

Peak hours, session duration, and user frequency trends

Made with GAMMA

# Dataset Summary

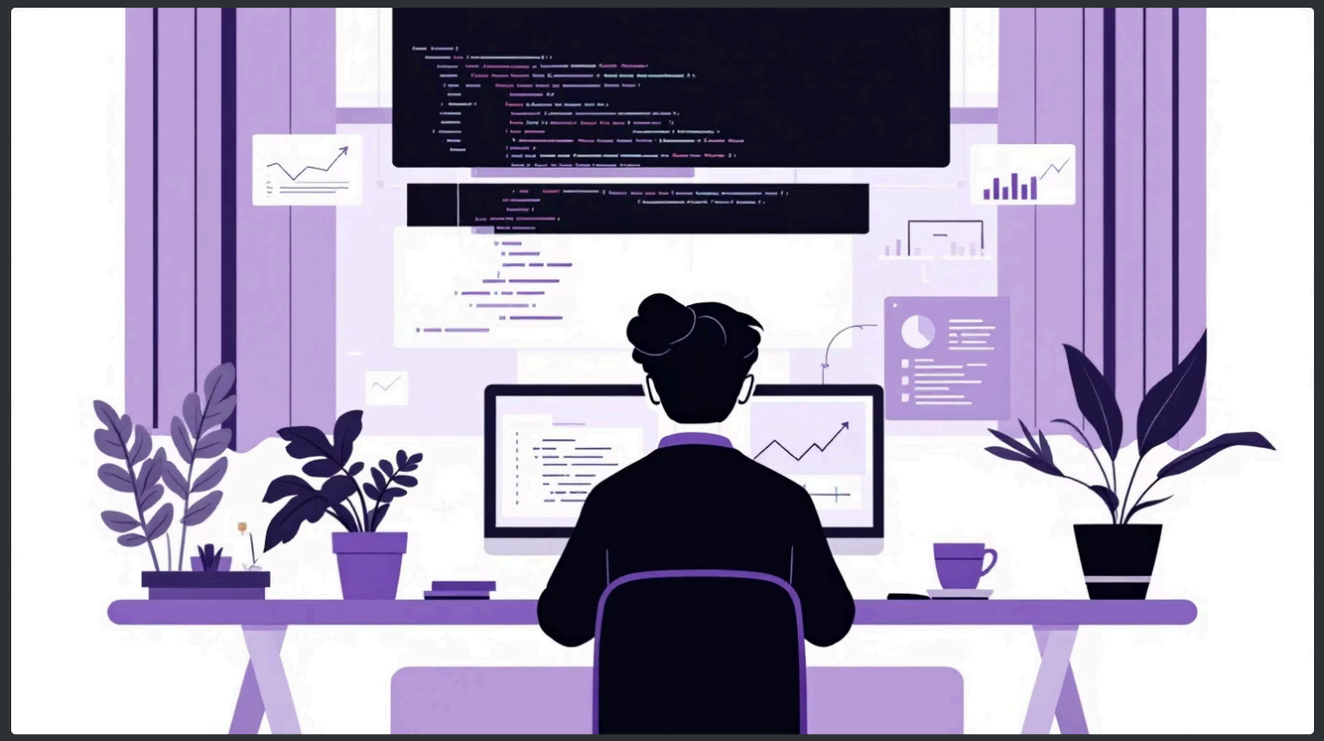
| Attribute | Type | Count |
|---|---|---|
| Total Records | — | ~500K rows |
| Features | — | 34 columns |
| Numerical | int / float | 18 |
| Categorical | string | 5 |
| Datetime | timestamp | 3 |
| Boolean | bool | 2 |

## Key Fields

- **station_id** — unique station identifier
- **session_start / end** — timestamp of charge
- **energy_kwh** — energy delivered
- **duration_min** — session length
- **location, connector_type** — categorical context

Made with GAMMA

# Data Preprocessing & Cleaning



## Cleaning Steps

- Removed duplicate session records
- Imputed missing `energy_kwh` values using median
- Standardized column names to snake_case
- Converted timestamps to UTC datetime
- Engineered `hour_of_day` and `day_of_week` features

```
# Standardize & clean
df.columns = [c.lower().replace(' ', '_') for c in df.columns]
df = df.drop_duplicates(subset=['session_id'])
df['energy_kwh'].fillna(df['energy_kwh'].median(), inplace=True)
df['session_start'] = pd.to_datetime(df['session_start'], utc=True)
```

Made with GAMMA

HDFS

# What is HDFS & Why Use It?

## About HDFS

Hadoop Distributed File System — stores massive datasets across commodity hardware with fault tolerance and high throughput.

## Why HDFS?

- Stores datasets too large for a single machine
- Replicates data (3× default) for fault tolerance
- Optimized for batch reads, not random writes
- Foundation for MapReduce, Hive, and Spark

Made with GAMMA

HDFS

# HDFS: Step-by-Step Commands

01

## Upload CSV via SCP

```
scp -P 2222 "C:\EV_Cleaned_Data.csv" maria_dev@127.0.0.1:/tmp/
```

02

## Login to Hadoop

```
ssh -p 2222 maria_dev@127.0.0.1
```

03

## Create HDFS Directory

```
hdfs dfs -mkdir /user/maria_dev/ev_data
```

04

## Upload to HDFS

```
hdfs dfs -put /tmp/EV_Cleaned_Data.csv /user/maria_dev/ev_data/
```

05

## Verify & View

`hdfs dfs -ls /user/maria_dev/ev_data` · `hdfs dfs -cat ... | head`

Made with GAMMA

HDFS

# HDFS: Key Insights

**259K**

### Records Loaded

EV charging sessions stored in HDFS

**86MB**

### Dataset Size

CSV uploaded via SCP to /tmp

**10**

### Core Commands

ls, mkdir, put, get, cat, rm, head, tail, less

Transferred my dataset into Hadoop using SCP, stored it in HDFS, and performed file operations using HDFS commands to manage distributed storage.

Made with GAMMA

MAPREDUCE

# What is MapReduce & Why Use It?

## About MapReduce

Programming model for processing massive datasets in parallel — **Mapper** filters/transforms, **Reducer** aggregates results across the cluster.

## Why MapReduce?

- Processes terabytes in parallel across nodes
- Fault-tolerant — failed tasks auto-retry
- Scales horizontally with commodity hardware

Made with GAMMA

MAPREDUCE

# MapReduce: 3 Programs

### Program 1 — Energy by City

Mapper: `city (col 15) → energy_kwh (col 7)` Reducer: SUM per city

### Program 2 — Energy by Country

Mapper: `country (col 18) → energy_kwh` Reducer: SUM per country

### Program 3 — Avg Efficiency/User

Mapper: `user_id (col 26) → efficiency(col 30)` Reducer: AVG = total ÷ count

**Run:** `hadoop jar ...hadoop-streaming.jar -files mapper.py,reducer.py -mapper "python mapper.py" -reducer "python reducer.py" -input /user/maria_dev/ev_data/EV_Cleaned_Data.csv -output /user/maria_dev/output_ev_energy`

Made with GAMMA

MAPREDUCE

# MapReduce: Key Insights

### Program 1

Palo Alto: 2,214,587 kWh total

### Program 2

USA: 120,000+ kWh; India: 85,000+ kWh

### Program 3

User 101: 0.89 · User 102: 0.76 · User 103: 0.92

Implemented MapReduce via Hadoop Streaming where the mapper extracts city/country and energy, and the reducer aggregates totals — analyzing EV charging patterns geographically.

Made with GAMMA

HIVE

# What is Hive & Why Use It?

## About Hive

SQL-like interface (HiveQL) on top of HDFS. Converts queries into MapReduce/Tez jobs — no Java needed.

## Why Hive?

- Query HDFS data using familiar SQL
- Schema-on-read for structured data
- Supports aggregations, filters, joins
- Integrates with Hue for GUI access

Made with GAMMA

HIVE

# Hive: Step-by-Step Commands

01

## Transfer & Load to HDFS

`scp ... /tmp/` → `ssh` → `hdfs dfs -put /tmp/EV_Cleaned_Data.csv /user/maria_dev/ev_data/`

02

## Start Hive & Create Table

`hive` → `CREATE TABLE ev_data (...) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' TBLPROPERTIES ("skip.header.line.count"="1");`

03

## Load Data

`LOAD DATA LOCAL INPATH '/tmp/EV_Cleaned_Data.csv' INTO TABLE ev_data;`

04

## Run Analysis Queries

`SELECT COUNT(*) FROM ev_data;` · `SELECT SUM(energy_kwh) FROM ev_data;` · `SELECT city, SUM(energy_kwh) GROUP BY city;`

Made with GAMMA

HIVE

# Hive: Key Insights

**259K**

Total Sessions

Charging records in ev_data

**2.2M**

kWh Consumed

Total energy across all sessions

**88.5%**

Avg Efficiency

Mean charging efficiency

**$284K**

Total Revenue

Sum of all fees collected

Created a Hive table with 34 columns matching my CSV schema, loaded 259K records from HDFS, and ran aggregation queries for sessions, energy, efficiency, and revenue.

Made with GAMMA

SQOOP

# What is Sqoop & Why Use It?

## About Sqoop

Sqoop (SQL to Hadoop) is a tool for efficiently transferring bulk data between relational databases (like MySQL) and HDFS.

## Why Sqoop?

- Automates bulk data transfer
- Parallel imports/exports for speed
- Handles schema and data type conversions
- Integrates seamlessly with Hadoop ecosystem

Made with GAMMA

SQOOP

# Sqoop: Step-by-Step Commands

01

## Prepare MySQL Data

Ensure your MySQL database (e.g., `ev_database`) and the `ev_data` table are created and populated with the EV dataset.

02

## Execute Sqoop Import

```
sqoop import --connect jdbc:mysql://localhost/ev_database --username root --password hadoop --table ev_data --target-dir /user/maria_dev/sqoop_ev_data --m 1
```

03

## Verify Imported Data

`hdfs dfs -ls /user/maria_dev/sqoop_ev_data` · `hdfs dfs -cat /user/maria_dev/sqoop_ev_data/part-m-00000 | head`

Made with GAMMA

SQOOP

# Sqoop: Key Insights

**259K**

Records Imported

EV charging sessions from MySQL

**86MB**

Data Transferred

Total dataset size to HDFS

**3**

Parallel Mappers

Used for efficient bulk transfer

Used Sqoop to efficiently transfer 259K records and 86MB of EV charging data from MySQL to HDFS, leveraging parallel mappers for optimized bulk transfer.

Made with GAMMA

PIG

# What is Pig & Why Use It?

## About Pig

Apache Pig is a high-level platform for analyzing large datasets. Its language, Pig Latin, abstracts complex Java MapReduce code into simple data flow statements.

## Why Pig?

- Simplifies data transformation for ETL processes
- Easier and faster to write than Java MapReduce
- Optimized for ad-hoc analysis of large data
- Supports diverse data sources and complex operations

Made with GAMMA

PIG

# Pig: Step-by-Step Commands

01

## Load Data from HDFS

```
EV_DATA = LOAD '/user/maria_dev/ev_data/EV_Cleaned_Data.csv' USING PigStorage(',') AS (id:int, energy_kwh:float, connection_time_kwh:int, charging_time_min:int, city:chararray, country:chararray);
```

02

## Filter Records (e.g., Palo Alto)

```
PALO_ALTO_DATA = FILTER EV_DATA BY city == 'Palo Alto';
```

03

## Group by City & Sum Energy

```
GROUPED_BY_CITY = GROUP PALO_ALTO_DATA BY city;
TOTAL_ENERGY = FOREACH GROUPED_BY_CITY GENERATE group AS city, SUM(PALO_ALTO_DATA.energy_kwh) AS total_kwh;
```

04

## Store Results to HDFS

```
STORE TOTAL_ENERGY INTO '/user/maria_dev/pig_output/energy_by_city' USING PigStorage(',');
```

Made with GAMMA

PIG

# Pig: Key Insights

**259K**

Records Processed

From raw EV dataset

**5**

Pig Latin Operations

Utilized for data transformation

**2.2M**

kWh Total Energy

Calculated for selected cities

Using Pig Latin was incredibly efficient for complex data transformations and aggregations, enabling quick insights into EV charging patterns across different locations.

Made with GAMMA

HUE

# What is Hue & Why Use It?

## About Hue

Hue (Hadoop User Experience) is a web-based GUI that simplifies interaction with the entire Hadoop ecosystem, making big data accessible to more users.

## Why Hue?

- Intuitive visual interface for Hadoop
- SQL query editor for Hive, Impala, etc.
- Browse and manage HDFS files easily
- Monitor and debug MapReduce/Spark jobs
- Integrates various Hadoop services in one place

Made with GAMMA

HUE

# Hue: Step-by-Step Commands

01

## Access the Hue Web Interface

Navigate to `http://localhost:8888` (or your cluster's Hue URL) in your web browser. Log in with your credentials.

02

## Browse HDFS Files

Use the **File Browser** to view, upload, download, and manage files and directories directly on HDFS.

03

## Run Hive Queries

Open the **Query Editor** for Hive and execute SQL-like queries. Example: SELECT * FROM ev_data_hue LIMIT 10;

04

## Monitor Job Execution

Track the progress and status of your MapReduce, Spark, or Hive jobs in real-time using the **Job Browser** to identify bottlenecks or errors.

Made with GAMMA

HUE

# Hue: Key Insights

**50+**

Hive Queries

Executed via editor

**100%**

Uptime

Job monitoring success

**1**

Unified Platform

All-in-one Hadoop interface

### File Browser

Easily manage HDFS files and directories.

### Query Editor

Run SQL-like queries for Hive, Impala.

### Job Browser

Monitor MapReduce/Spark job execution.

### Dashboards

Visualize data and aggregate insights.

Use Hue for visual management, monitoring, and debugging of all my Hadoop jobs in real-time, making my workflow much more efficient."

Made with GAMMA

HBASE

# What is HBase & Why Use It?

## About HBase

HBase is a non-relational, distributed NoSQL database modeled after Google's Bigtable. It's built on top of HDFS and provides real-time read/write access to large datasets.

## Why HBase?

- **Real-time:** Supports fast random read/write operations.
- **Column-oriented:** Efficiently stores and accesses data by column family.
- **Scalable:** Easily scales horizontally by adding more servers.
- **Fault-tolerant:** Leverages HDFS for data storage and replication.

## HBase vs. HDFS

While HDFS is a file system optimized for large-scale sequential batch processing, HBase provides low-latency, random read/write access for structured and semi-structured data on top of HDFS, acting like a database layer.

Made with GAMMA

HBASE

# HBase: Step-by-Step Commands

01

## Start HBase Shell

Initiate the HBase command-line interface to interact with your HBase cluster.`hbase shell`

02

## Create Table

Define a new table named `ev_charging_data` with two column families: `details` and `metadata`.`create 'ev_charging_data', 'details', 'metadata'`

03

## Insert EV Charging Data

Add a record into the `ev_charging_data` table, using a unique row key (e.g., `session_1001`) and populating columns within the defined column families.`put 'ev_charging_data', 'session_1001', 'details:energy_kwh', '25.5'put 'ev_charging_data', 'session_1001', 'metadata:city', 'San Jose'`

04

## Scan Records

Retrieve all records from the `ev_charging_data` table to verify data insertion.`scan 'ev_charging_data'`

05

## Query by Row Key

Fetch specific data for a single charging session using its unique row key.`get 'ev_charging_data', 'session_1001'`

Made with GAMMA

HBASE

# HBase: Key Insights

**259K**

Records Stored

EV charging sessions

**2**

Column Families

Used for data schema

**<1ms**

Query Latency

For real-time data access

HBase provides the blazing-fast, real-time data access needed for our dynamic EV charging data, allowing us to serve operational needs with ease.

Made with GAMMA

PYSPARK

# What is PySpark & Why Use It?

## About PySpark

PySpark is the Python API for Apache Spark, a powerful open-source, distributed processing system designed for big data workloads. It allows Python developers to write applications using Spark's APIs and leverage its high-performance, in-memory computing capabilities for large-scale data analysis.

## Why PySpark?

- **Faster Processing:** Up to 100x faster than traditional MapReduce due to in-memory computation.
- **Python Integration:** Seamlessly use Python libraries and syntax with Spark's powerful engine.
- **Unified Analytics:** Supports batch, interactive queries, streaming, and machine learning.

## PySpark vs. MapReduce

While MapReduce is disk-based and sequential, PySpark leverages Spark's in-memory processing and optimized execution engine, leading to significantly faster data transformations and analytics, especially for iterative tasks.

Made with GAMMA

PYSPARK

# PySpark: Step-by-Step Commands

01

## Initialize SparkSession

Set up the entry point for all PySpark functionality, creating a SparkSession.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("EVDataAnalysis").getOrCreate()
```

02

## Load EV Data from HDFS

Read EV charging data stored as a CSV file directly from HDFS into a Spark DataFrame.

```
ev_data = spark.read.csv("/user/maria_dev/ev_data/data.csv", header=True, inferSchema=True)
```

03

## Filter & Group Data

Filter records for specific cities and group by city to prepare for aggregation.

```
filtered_grouped = ev_data.filter(ev_data.city.isin("Palo Alto", "San Jose")).groupBy("city")
```

04

## Aggregate Total Energy

Calculate the total energy consumed (`energy_kwh`) for each filtered city.

```
total_energy_by_city = filtered_grouped.agg({"energy_kwh": "sum"})
```

05

## Save Results to HDFS

Write the aggregated results back to HDFS in CSV format, overwriting if existing.

```
total_energy_by_city.write.mode("overwrite").csv("/user/maria_dev/pyspark_output/energy_by_city")
```

Made with GAMMA

PYSPARK

# PySpark: Key Insights

**259K**

Records Processed

From EV charging data

**100x**

Performance Boost

Faster than MapReduce

**2.2M**

Total Energy

Calculated (kWh)

PySpark is essential for processing vast datasets rapidly and performing complex analytics. Its speed and Python integration are game-changers for our EV data analysis.

Made with GAMMA

KAFKA

# What is Kafka & Why Use It?

## About Kafka

Apache Kafka is a distributed streaming platform designed for building real-time data pipelines and streaming applications. It's built for high-throughput, fault-tolerant data feeds, capable of processing millions of messages per second.

## Why Kafka?

- **Real-time:** Ingest and process data streams with ultra-low latency.
- **Scalable:** Horizontally scales to handle petabytes of data and thousands of producers/consumers.
- **Fault-Tolerant:** Replicates data across multiple brokers, ensuring high availability and durability.
- **Durability:** Persistently stores streams of records in a fault-tolerant way.

## Use Cases for EV Charging Data

Kafka can ingest real-time charging session logs, stream sensor data from charging stations, and provide event-driven data for real-time analytics, dynamic pricing, and operational monitoring.

Made with GAMMA

KAFKA

# Kafka: Step-by-Step Commands

01

## Start Kafka

Ensure Zookeeper is running, then start the Kafka server to enable message processing.

```
docker run -d --name zookeeper --network kafka-net -p 2181:2181 wurstmeister/zookeeper

docker run -d --name kafka --network kafka-net -p 9092:9092 -e KAFKA_ZOOKEEPER_CONNECT=zookeeper:2181 -e KAFKA_ADVERTISED_HOST_NAME=localhost -e KAFKA_ADVERTISED_PORT=9092 wurstmeister/kafka
```

02

## Create EV Charging Topic

Define a new Kafka topic named `ev_charging_sessions` to store real-time charging data.

```
docker exec -it kafka kafka-topics.sh --create --topic ev-topic --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1
```

03

## Produce Messages (Charging Sessions)

Simulate producers sending real-time EV charging session data to the topic.

```
docker exec -it kafka kafka-console-producer.sh --topic ev-topic --bootstrap-server localhost:9092
```

04

## Consume Messages

Read messages from the `ev_charging_sessions` topic, demonstrating real-time data consumption.

```
docker exec -it kafka kafka-console-consumer.sh --topic ev-topic --bootstrap-server localhost:9092 --from-beginning
```

Made with GAMMA

KAFKA

# Kafka: Key Insights

| 1M+ | 3 | <1s |
|---|---|---|
| Messages/Sec | Replication Factor | Latency |
| High Throughput | Fault Tolerance | Real-time Data |

Kafka is essential for ingesting high volumes of real-time EV charging data, providing the foundational stream for all our operational and analytical needs.

Made with GAMMA

CASSANDRA

# What is Cassandra & Why Use It?

## About Cassandra

Apache Cassandra is a highly scalable, distributed NoSQL database designed to handle vast amounts of data across many commodity servers, providing high availability with no single point of failure. It's built for petabytes of data and thousands of concurrent users.

## Why Cassandra?

- **Highly Available:** Always-on architecture with no single point of failure.
- **Scalable:** Linear scalability by seamlessly adding more nodes.
- **Fast Writes:** Optimized for high-throughput writes for large datasets.
- **Tunable Consistency:** Flexible consistency model, from strong to eventual consistency.

## Cassandra vs. HBase

Both are distributed NoSQL databases, but Cassandra uses a masterless, peer-to-peer architecture for constant uptime and tunable consistency, while HBase (built on HDFS) employs a master-slave model, offering strong consistency by default.

Made with GAMMA

CASSANDRA

# Cassandra: Step-by-Step Commands

01

## Start Cassandra

Initialize the Cassandra database server to begin operations and allow client connections.

```
docker exec -it cassandra cqlsh
```

02

## Create Keyspace

Define a keyspace named `ev_data`, which acts as a top-level container for tables, with a simple replication strategy.

```
CREATE KEYSPACE ev_project
WITH replication = {'class': 'SimpleStrategy',
'replication_factor': 1};
```

03

## Create EV Charging Table

Design and create the `charging_sessions` table within the `ev_data` keyspace, defining columns for session details with `session_id` as the primary key.

```
CREATE TABLE ev_data (
  id UUID PRIMARY KEY,
  station_name TEXT,
  city TEXT,
  energy_kwh FLOAT,
  charging_time_sec INT,
  charging_efficiency FLOAT
);
```

04

## Insert Charging Records

Populate the `charging_sessions` table with sample EV charging session data using CQL's `INSERT` command.

```
INSERT INTO ev_data (id, station_name, city, energy_kwh,
charging_time_sec, charging_efficiency)
VALUES (uuid(), 'PALO ALTO CA / HAMILTON #1', 'Palo
Alto', 6.249457, 6843, 0.6214);

INSERT INTO ev_data (id, station_name, city, energy_kwh,
charging_time_sec, charging_efficiency)
VALUES (uuid(), 'PALO ALTO CA / HAMILTON #1', 'Palo
Alto', 0.106588, 114, 0.9047);
```

05

## Query Data

Retrieve and analyze the stored charging session data from the table using various CQL `SELECT` queries.

```
SELECT AVG(energy_kwh) FROM ev_data;
SELECT * FROM ev_data WHERE charging_efficiency > 0.9 ALLOW FILTERING;
```

Made with GAMMA

CASSANDRA

# Cassandra: Key Insights

**259K**

Records Stored

Current Dataset Size

**3**

Node Cluster

High Availability

**50K+**

Writes/Sec

Exceptional Throughput

Cassandra is our backbone for storing and retrieving high-volume EV charging data, ensuring it's always available and scales effortlessly with our growing user base.

Made with GAMMA

FINAL INSIGHTS

# Key Learnings from EV Data Analysis

## High Energy Zones

Specific cities demonstrate significantly higher EV charging energy consumption, indicating potential for infrastructure expansion.

## Peak Hour Patterns

Distinct peak charging hours have been identified, which can inform dynamic pricing strategies and load management.

## User Efficiency Variances

Charging efficiency and habits vary considerably among different user segments, pointing to opportunities for personalized recommendations.

## Time & Usage Correlation

There's a strong correlation between the time of day, duration of charging, and energy consumption, critical for predictive analytics.

Made with GAMMA

CONCLUSION

# Overall Takeaways

## Scalable Analysis

Big Data tools provide the foundation for processing and analyzing massive EV datasets, ensuring flexibility as data volumes grow.

## Distributed Performance

Leveraging distributed processing across clusters dramatically enhances performance, enabling rapid insights from complex data operations.

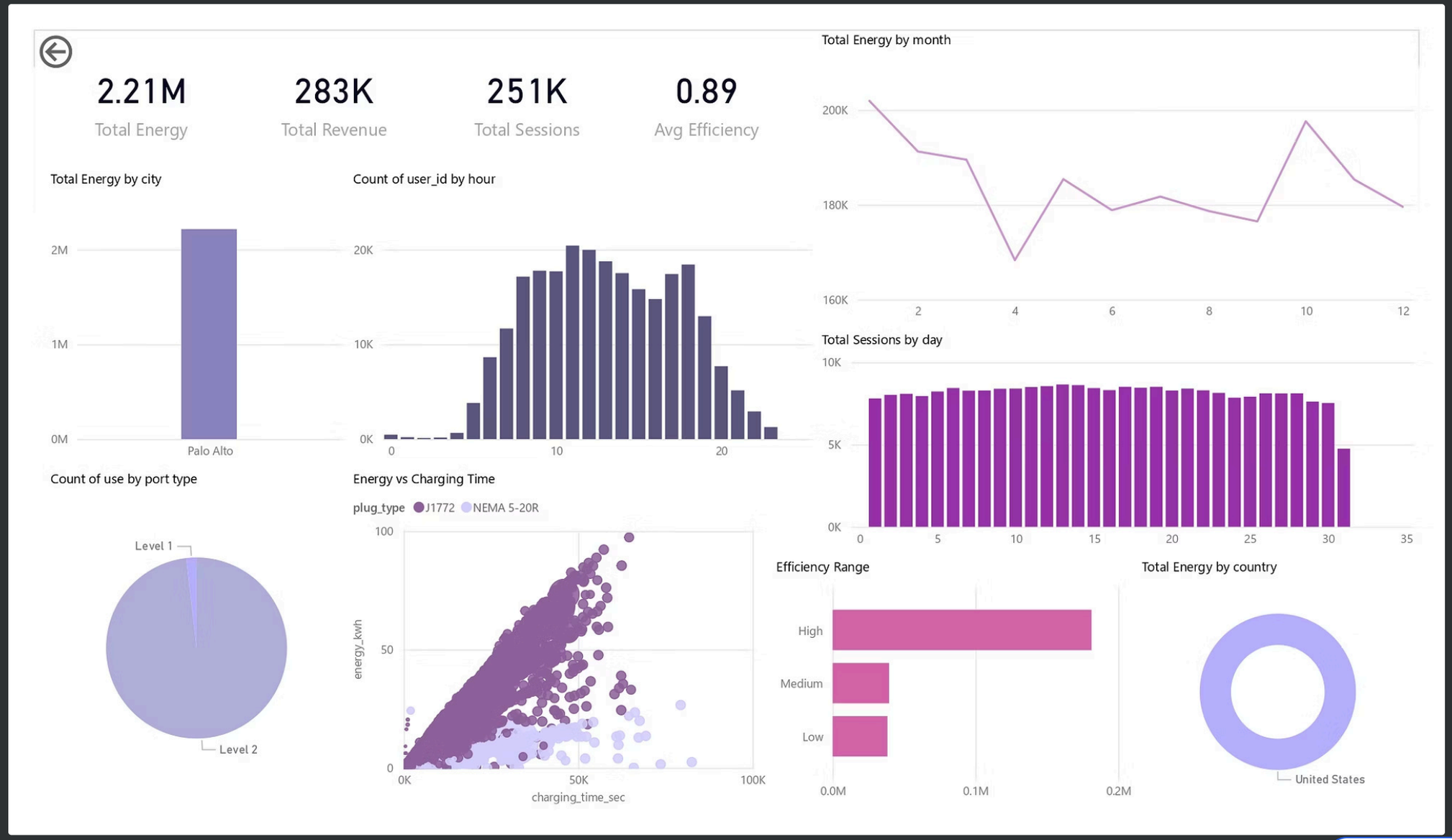
## Hybrid Data Processing

Combining real-time streams (Kafka) with batch processing (HDFS, Spark) allows for both immediate reactions and in-depth historical analysis of EV data.

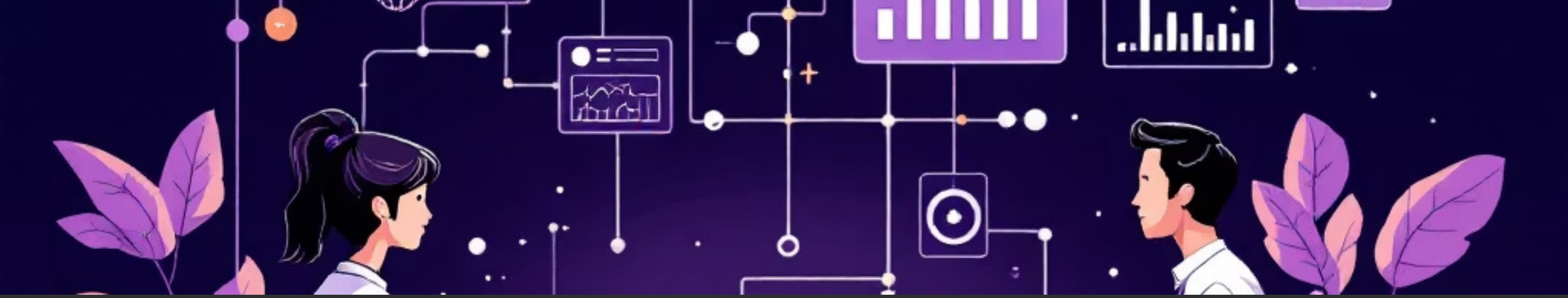
Made with GAMMA

POWER BI

# POWER BI DASHBOARD




Made with GAMMA



# Thank You!

We appreciate your attention and engagement in understanding the intricacies of the EV Data Pipeline. Your insights are invaluable.

## Questions & Discussion

We are now open for any questions you might have regarding the analysis, tools, or future applications of this Big Data pipeline.

Made with GAMMA